

Hein Zarni Aung

Computer Science + Applied Math & Statistics

☎ +1-(917)-854-7709 ✉ heinzarniaung.dev@gmail.com 🔗 LinkedIn 🔗 GitHub

Cover Letter

Dear Hiring Team,

I came across StudyFetch's research on how students use their AI tutor. Most of it goes toward concept explanations and summarizing, and direct-answer requests drop off sharply as students get more comfortable with it. That struck a chord, because it's basically what I spent last semester trying to get right as a TA: the difference between a student learning something and a student just getting unstuck for five minutes. That's not something I expected to find in a job posting, but it's the reason this internship feels like the right room to be building in.

Most of my time these days goes into two internships that don't overlap much. At Clustr, I've spent the past several months building React Native features and the Firebase Cloud Functions backend underneath them, handling CRUD logic, permissions, and activity feeds. It's the stuff a user never sees, but it breaks everything if I get it wrong. At Prenora Dynamics, I own an in-platform video chat feature end-to-end. I evaluate SDKs, decide what to build versus buy, and set up the Postgres/Supabase backend for a separate scheduling app, with nobody telling me the right answer. Between the two, I've gotten comfortable moving across JavaScript, TypeScript, and Python day to day, which is close to the stack this role uses.

However, the project I'd actually bring up if someone asked me about interesting work is TuneAcademy, a hackathon build that matches music students with instructors by analyzing their playing. I built the AI/audio pipeline using Essentia and NumPy for pitch and rhythm features, plus ElevenLabs to generate feedback reports. When a full ML model turned out to be unrealistic in a 36-hour window, I pivoted to a rule-based scoring system that actually shipped. I also set up CI with GitHub Actions so it didn't fall apart on deploy. It's the closest thing I've built to an AI feature that has to be fast and reliable, not just a demo, and it's the project I learned the most from.

That TA work has stuck with me more than almost anything else I've done. Running recitations for 240 students meant figuring out, in real time, which explanation would actually land for someone stuck on recursion for the third week in a row. I've spent this past year doing something similar one-on-one, mentoring international students adjusting to life here. Both taught me that the easy path, just handing someone the answer, is usually the wrong one. That's the mindset I bring to the code I write and the problems I try to solve.

Thank you for the opportunity, and I'll be looking forward to hearing back from you.

Sincerely,
Hein Zarni Aung